

```

1 %=====
2 % Lucas Tree Model (Octave)
3 %=====
4
5 clear
6 clc
7
8 pkg load statistics %For Octave
9
10 %LaTeX Interpreter if ==1 (requires LaTeX to be installed)
11 latex_interpreter=0;
12
13 gamma=1.0;      %Coefficient of Constant Risk Aversion
14 beta=0.95;      %Discount Factor
15 alpha=0.9;      %\log y_{t+1}= \alpha \log y_t + \sigma \varepsilon_t
16 sigma=0.1;      %Standard deviation for the Log-Normal Distribution
17
18 %Grid for y
19 %Since we cannot just have a state space [0,\infty), we consider [y_min, y_max]
20 ssd=sigma/sqrt(1-alpha^2);      %The standard deviation for \log y_t
21 y_min=exp(-4*ssd);      %The minimum of the grid
22 y_max=exp(4*ssd);      %The maximum of the grid
23 dim_y=100;      %Grid size for the vector of y
24 y_vec=linspace(y_min,y_max,dim_y); %Grid for y
25
26 %Realizations of y_t (to compute/approximate integrals)
27 %Two equivalent ways
28 %epsilon: (1,500)-vector, iid standard normal
29 %r_ln: (1,500)-vector, iid log-normal with (mean,sd)=(0,sigma)
30 %rng("default"); %Initialize the random number generator
31 %For Octave rng("default") instead of rng('default')
32 epsilon=normrnd(0,1,1,500);
33 rng("default"); %Initialize the random number generator
34 %For Octave rng("default") instead of rng('default')
35 r_ln=lognrnd(0,sigma,1,500);
36 %Note: r_ln=exp(sigma*epsilon);
37 % y_{t+1} = y^{\alpha}_t \exp ( \sigma \varepsilon_t )
38
39 %h(y) = beta* \int u'(G(y,z)) G(y,z) \phi(dz)
40 h=zeros(1,dim_y);
41 for i=1:dim_y
42     h(i) = beta* mean(((y_vec(i)^alpha).*r_ln).^(1-gamma));
43 end
44
45 f=zeros(1,dim_y); %f
46 f_new=zeros(1,dim_y); %Tf
47
48 max_iter=2000; %The Maximum Number of Iterations
49 toler=0.1^4; %Tolerance Value
50 converge=0;
51 % converge is a binary variable, where it takes 1 when the value
52 % function iteration ended within the maximum number of rounds (max_iter).
53
54 tic %start stopwatch timer (optional)
55 for ell=1:max_iter
56     for i=1:dim_y
57         f_new(i) = h(i) + beta* mean( interp1(y_vec,f,(y_vec(i)^alpha).*r_ln,'spline', 'extrap' ) );
58         %For Octave, 'extrap' added (otherwise interp1 may return NaN)

```

```

59     end %end of for i=1:dim_N
60     %
61     if norm(f_new-f,Inf)<toler %alternative: if max(abs(f_new-f))<toler
62         converge=1;
63         break;
64     end %end of if
65     %
66     f=f_new;
67 end
68 comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)
69
70 price=f.*y_vec.^gamma;
71
72 %Convergence
73 if converge==1
74     fprintf('The value function iteration converged with %d iterations.\n', ell);
75     fprintf('The value function iteration took %f seconds.\n', comp_time);
76 else
77     fprintf('The value function iteration did not converge.\n');
78 end
79
80 figure()
81 hold on
82 plot(y_vec,price,'LineWidth',2);
83 if gamma==1
84     plot(y_vec,(beta/(1-beta))*y_vec,'--','LineWidth',2);
85 end
86 grid on
87 hold off
88 xlim([y_min, y_max]);
89 if latex_interpreter==1
90     xlabel('$y$', 'Interpreter', 'latex', 'FontSize', 12);
91     ylabel('$p$', 'Interpreter', 'latex', 'FontSize', 12);
92     title('Equilibrium Pricing Function $p$', 'Interpreter', 'latex', 'FontSize', 14);
93 else
94     xlabel('y', 'FontSize', 12);
95     ylabel('p', 'FontSize', 12);
96     title('Equilibrium Pricing Function p', 'FontSize', 14);
97 end
98

```